T6 - You don't know JS - II

5 March - 2026

Day-22

8 - methods - of - primitives.JS

// It all Come ~~at~~ Got
// Object are heaviour than Primitive

String - Datatype
String - wrapper Object
Number → wrapper Object

```
let str = "ak Cool";
let strClean = str.toLowerCase();
Consolelog (strClean)
```

9 - number JS

```
let million = 1_000_000;
// e → 10
let lakh = 1e5;
    ↳ 1 × 10^5
let cr = 4e7;
// e2 → 100
// e-2 → 10^-2 (10^-2)
let dAtom = 5e-10
let dNucleus = 1e-15;
```

// Octal
Consolelog (0o13)

// Hexa
Consolelog (0x255);
Consolelog (0xff);

// Binary
Consolelog (0b111); 111 → 4+2+1=7

// toString

console.log((11).toString(16));

10 - String.js

```
let single = 'single-quoted';

let double = "double-quoted";

let backticks = `backticks (string-interpolation)`

let fancy = "this is \"me\"";

console.log(fancy);

let windowDownfall = `Downfall started from
    * INS
    * W10
    * W11 AI
    * Copilot
    * Patched bug
    * Blue Screen of death
`

console.log(windowDownfall)

// length
// charAt
// Access
// [], .pos()

console.log(windowDownfall[0])

// console.log(windowDownfall[windowDownfall.length-1])

console.log(windowDownfall.charAt(-1))
```

↳ h

// iterable
// for...of

```
for (const char of 'meow') {
    console.log(char);
}
```

// String are immutable

```
let str = "Rama";
~~str = "R" + str[1]~~
str = "R" + str.substring(1);
console.log(str);
```

↳ Rama

// - array.js

```
let arr1 = new Array();
```

// Array in JS is → Deque ... [Stack + queue]
// queue [first in first out] → push, shift ...
// Stack [LIFO] → Last in first out → push, pop

// Internals
// obj but special
// Index, value
// The JS engine tries to store its elements in the contiguous memory area, and offer random

// Loops

```
for (let i=0; i< appleTvShows.length; i++){
    // Console.log (appleTvShows[i]);
}

for (Const tvShow of appleTvShows){
    Console.log (tvShow);
}
```

// Length - Property

```
let food = [];
food [3] = "Idli";
Consolelog food.length; // 8
```

// DSA array . fill (true)

```
Console.log (new Array (50).fill (0));

Console.log {[] == []}
                -> Error
```

// Methods

// Splice (), Concat ()

// forEach () → update existing array

// Search

// indexOf (), lastIndexOf (), includes (),
find (), findIndex (), findLastIndex()

// filter (), we count to find multiple elements

// Transform

// reverse () → update existing array

// Split (delim), Join (glue)

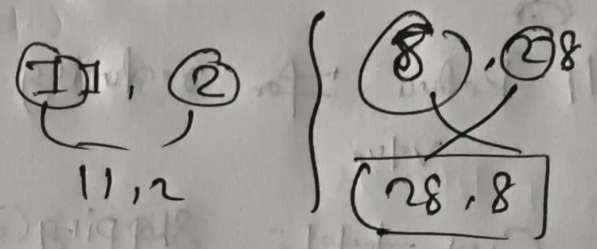
// Sort ()

```
Const number = [5, 11, 2, 37, 41, 77, 8, 28];

Console.log (number.sort()); [11, 2, 28, 37, 41, 5, 77, 8]
```

// Lexicographically

↳ only 1st Character Compare




```
function CompareNumber (a, b) {
    if (a > b) return 1;
    if (a < b) return -1;
    if (a === b) return 0;
}

Console.log (numbers.sort (ComparNumeric));
```

↳ [2, 5, 8, 11, 28, 37, 41, 77]

```
Console.log (number.sort ((a, b) => a - b));

Console.log (names.sort ((a, b) => a.localeCompare(b)))
```

```
Const ShoppingCart = [
  {name: 'Macbook pro 16', price = 3499},
  {name: 'iPad 17 pro', price: 1099},
  {name: 'iPad pro', price: 1299},
  {name: 'Apple watch', price: 249}
];
Const price = ShoppingCart.map(item => item.price =>
  Console.log(price))

// Reduce for reducing into single value
Const total = ShoppingCart.reduce((total, price) =>
  total + price, 0)
Console.log(total);

Const n = [20,

  Const max = n.reduce((max, curr) => {
    if (max < curr) max = curr;
    return max;
  }, -Infinity);

  Const min = n.reduce((min, curr) => {
    if (min > curr) min = curr
    return min;
  }, Infinity)
```

11- itrable.js

// That is a concept that allow us to make any object useable in a for... of loop

// String, Array..

// [Symbol.iterator] helps to decide where it's iterable.

↳ it decides if that object iterable or not

// this is a method

// it returns a object

// that object has next() method

// next() return {done: Boolean, value: any}

// Let's make obj iterable

```
let playlist = {
    songs: ["My Sweet Lord", "Hyaro Vaidhvar", "Sunrise",
            "Like a River"],
    from: 0
};

~~playlist[Symbol.iterator] = function~~ ()
    ~~return {~~
        ~~next()~~
            ~~return {done: true};~~
    ~~}~~
}
```

```
[Symbol.iterator]: function() {
    let curr = this.from;
    const args = this.args;

    return {
        next() {
            if (curr < arg.length) {
                return {done: false, value: arg[curr++]};
            }
            return {done: true}
        }
    };
```

13- array-like JS

14-map.js

// Map → obj (key can be anything)

// In obj key can be either string or symbol.

17-date-time.js

// In JS Date always has time → date & time

// Create

```
const now = new Date();

console.log(now)

const pizzaBDay = new Date('2001-01-01');
```

or

```
new Date(200, 1, 1, 8, 32);
```

// getFullYear(), getMonth(), getDate(), getHours()
// getMilliseconds(), getDay(), getMinutes()

// Setting Date Components.

- setFullYear(year, [month], [date]);
- setMonth(month, [date]);
- setDate(date)
- setHours(hour, [min], [sec], [ms])